

📖 Super Easy C++ Notes (Hinglish + Clear + With Output)

1) Inline Function

📖 **Definition:** Inline function ka matlab hota hai ki function ka code seedha copy ho jata hai call wali jagah pe. Isse program fast hota hai.

Code:

```
#include <iostream>
using namespace std;

inline int square(int x) {
    return x * x;
}

int main() {
    cout << square(5);
}
```

Output:

25

2) Recursive Function

📖 **Definition:** Recursive function apne aap ko call karta hai. Base case hona zaroori hai.

Code:

```
#include <iostream>
using namespace std;

int fact(int n) {
    if(n == 0) return 1; // base case
    return n * fact(n-1);
}

int main() {
    cout << fact(5);
}
```

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 int fact(int n) { 5 if(n == 0) return 1; // base case 6 return n * fact(n-1); 7 } 8 9 int main() { 10 cout << fact(5); 11 } 12</pre>	120 === Code Execution S

Output:

120

CODE → [CLICK ME](#)

3) Function Pointer

🔗 **Definition:** Function pointer ek pointer hai jo function ka address store karta hai.

Code:

```
#include <iostream>
using namespace std;

void hello() { cout << "Hello"; }

int main() {
    void (*fp)() = hello; // function pointer
    fp();
}
```

Output:

Hello

ANOTHER →

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 void hello() { 5 cout << "Hello" << endl; 6 } 7 8 int main() { 9 void (*fp)() = hello; // function pointer 10 11 cout << "Function ka address: " << (void*)fp << endl; 12 13 14 fp(); // calling function using pointer 15 return 0; 16 }</pre>	<pre>Function ka address: 0x401156 Hello === Code Execution Successful ===</pre>

CODE→

[CLICK ME](#)

4) Multi-Dimensional Array

📖 **Definition:** Multi-D array ka matlab hota hai matrix type data (rows × cols).

Code:

```
#include <iostream>
using namespace std;

int main() {
    int arr[2][2] = {{1,2},{3,4}};
    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            cout<<arr[i][j]<<" ";
        }
        cout<<endl;
    }
}
```

Output:

```
1 2
3 4
```

CLICK - > [CODE](#)

5) String in C++

📖 **Definition:** String ek special class hai jo text store aur modify karne ke liye use hoti hai.

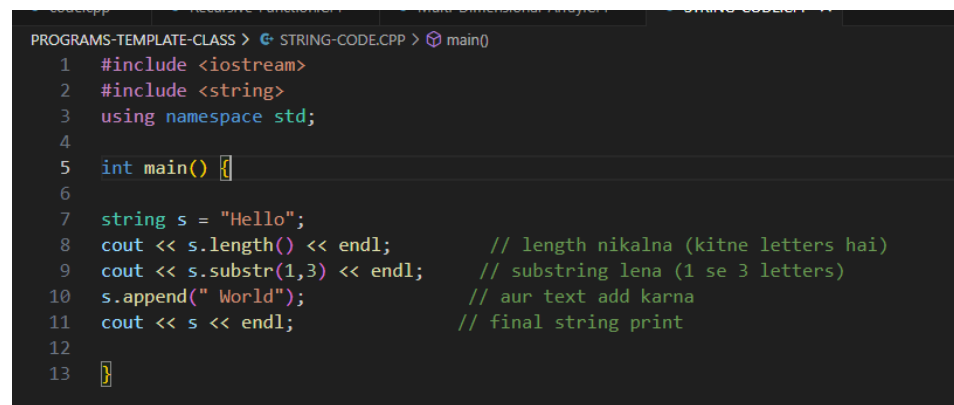
Code:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s = "Hello";
    cout << s.length() << endl; // length
    cout << s.substr(1,3) << endl; // substring
    s.append(" World");
    cout << s << endl;
    return 0;
}
```

Output:

```
5
ell
Hello World
```

A screenshot of a C++ code editor with a dark theme. The code is for a program that demonstrates string operations. It includes the necessary headers, uses the std namespace, and defines a main function. Inside main, a string 's' is initialized to "Hello". Then, the length of the string is printed (5), followed by a substring of length 3 starting from index 1 (which prints "ell"). The string is then appended with " World", and the final string is printed. Comments in Hindi explain each step: "length nikalna (kitne letters hai)", "substring lena (1 se 3 letters)", "aur text add karna", and "final string print".

```
PROGRAMS-TEMPLATE-CLASS > STRING-CODE.CPP > main()
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  int main() {
6
7      string s = "Hello";
8      cout << s.length() << endl; // length nikalna (kitne letters hai)
9      cout << s.substr(1,3) << endl; // substring lena (1 se 3 letters)
10     s.append(" World"); // aur text add karna
11     cout << s << endl; // final string print
12
13 }
```

CLICK ME → [CODE](#)

6) Goto Statement

📖 Definition:

Goto ek jump statement hai jo seedha label par le jata hai. Use karna recommend nahi hai.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n = 1;
    start:
    cout << n << " ";
    n++;
    if(n<=5) goto start;
}
```

Output:

1 2 3 4 5

```
DGRAMS-TEMPLATE-CLASS > goto-statement.cpp >
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n = 1;
6      start:
7
8      cout << n << " ";
9      n++;
10
11      if(n<=5) goto start;
12  }
13
```

Click me –[code](#)

7) Templates

☞ **Definition:** Template se ek hi code different data types ke liye kaam karta hai.

(a) Function Template

```
#include <iostream>
using namespace std;

template <typename T>

T add(T a, T b) {
    return a+b;
}

int main() {
    cout << add(5,6) << endl;
    cout << add(2.5,3.5) << endl;
}
```

Output:

11
6

main.cpp	Run	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 template <typename T> 5 T add(T a, T b) { 6 return a+b; 7 } 8 9 int main() { 10 cout << add(5,6) << endl; 11 cout << add(2.5,3.5) << endl; 12 cout << add(2.55,13.5) << endl; 13 } 14</pre>		<pre>11 6 16.05 === Code E</pre>

Code -> [Click me](#)

(b) Class Template

✔ Template in C++ (Definition)

☞ Template ek **blueprint** hota hai jisme hum **generic code** likhte hai, jo alag-alag data types (int, float, double, string...) ke liye automatically kaam karta hai **without code duplicate**.

```
PROGRAMS-TEMPLATE-CLASS > class-template-1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  template <class T>
5  T add(T a, T b) {
6      return a + b;
7  }
8
9  int main() {
10     cout << add(5, 10) << endl;    // int
11     cout << add(2.5, 3.5) << endl; // double
12     cout << add(string("Hi "), string("Bhai")) << endl; // string
13 }
14
```

Code—> [click me](#)

Output:

10